GrowBuddy — Database Tables and Relations

PostgreSQL 18, schema owned by Alembic at revision `0007`. Drawn from `backend/app/models.py` and `backend/app/database.py` on 2026-09-27. When a migration changes a table, update this file in the same pass.

17 tables and 1 view. The diagrams are split by area because one diagram with every table and every foreign key is unreadable; the overview comes first.

Reading the diagrams:

- `||--o{` one to many, `||--o|` one to zero-or-one, `|o--o{` optional parent (the foreign key is nullable).
- `PK` primary key, `FK` foreign key. Unique columns, and a key that is also a foreign key, are noted in the column's comment instead: older Mermaid builds (as bundled in some editors) reject `UK` and `PK, FK`.
- Readable ids (`U_000001`, `TR_…`, `CL_…`, `ST_…`, `S_…`) are handed out by `id_counters`; every such table also carries a random `uuid`.

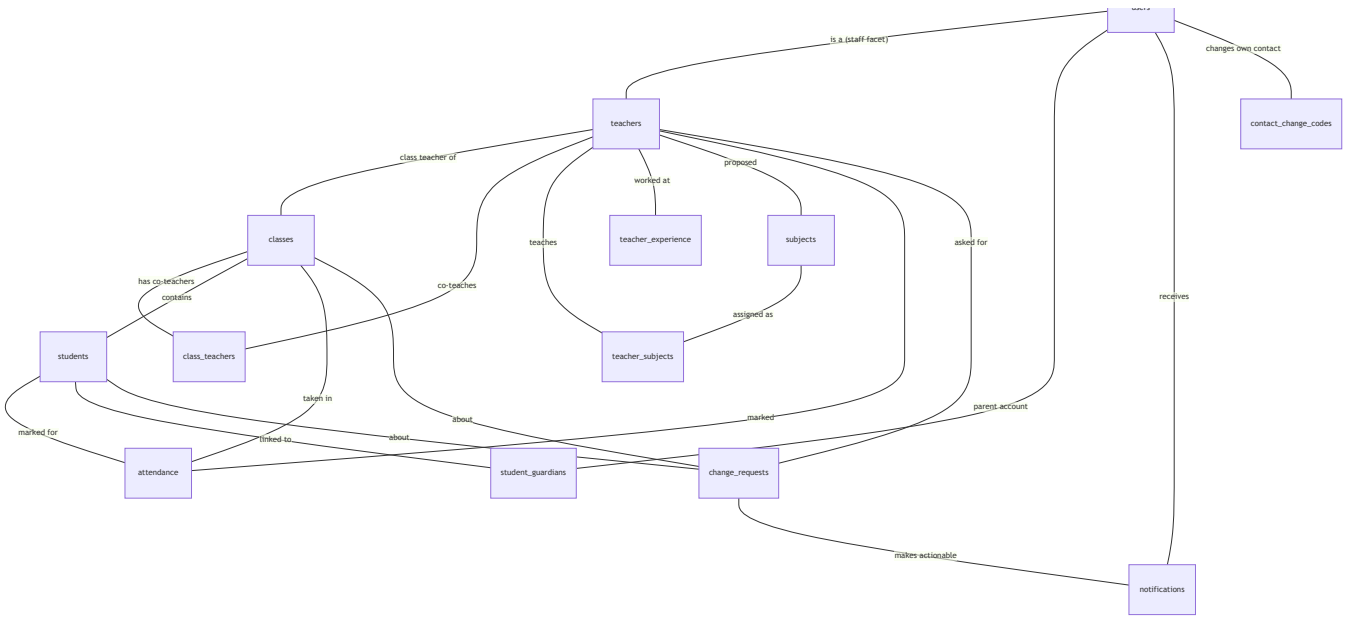
---

1. Overview

Every table and how they connect. Columns are in the sections below.

Refer the diagram below



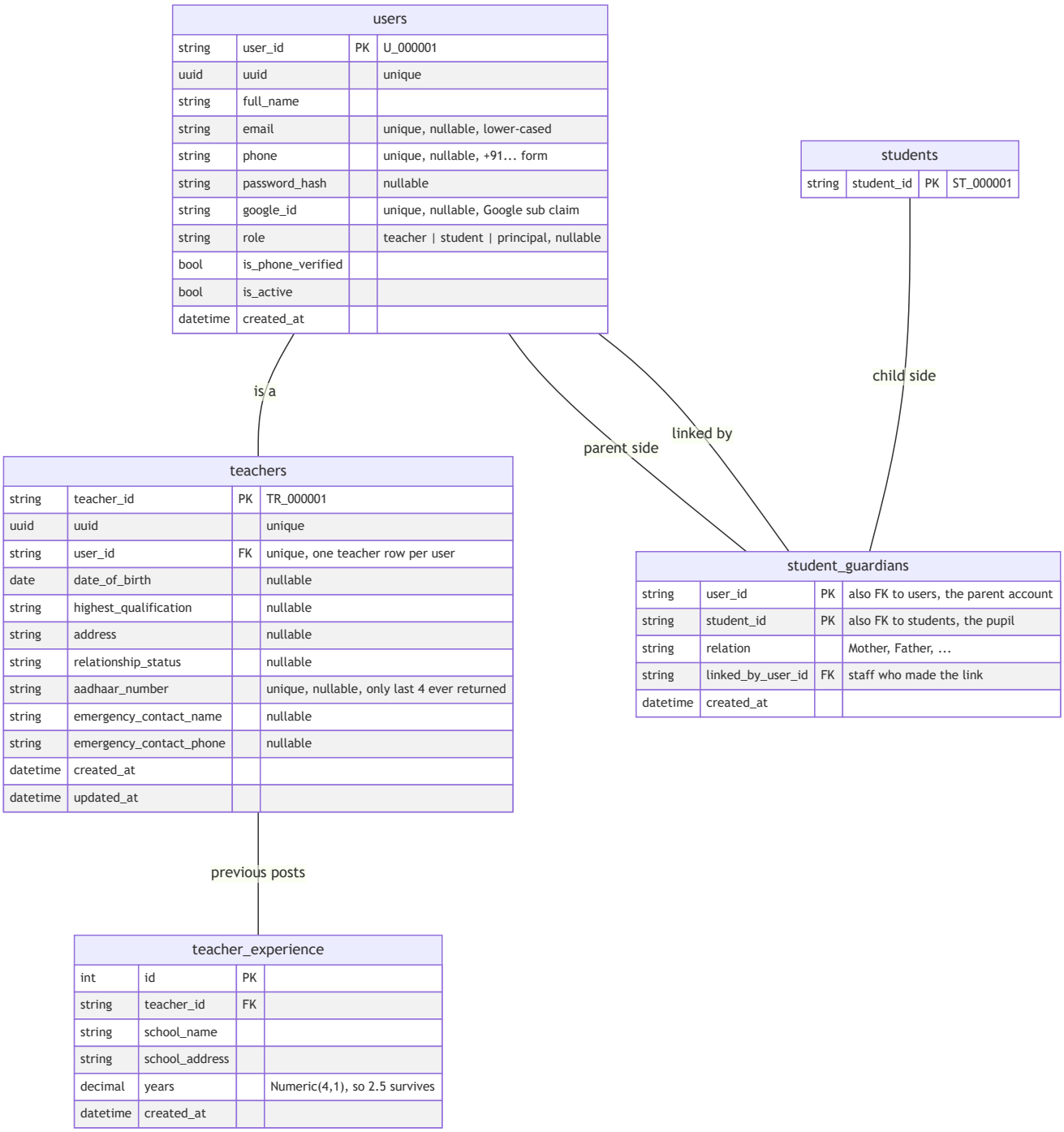


Standalone, with no foreign keys: `id_counters`, `otp_codes`, `password_reset_codes`. The two code tables key off a bare phone or email because a code is often issued before the account exists.

---

2. People — accounts, staff and parents

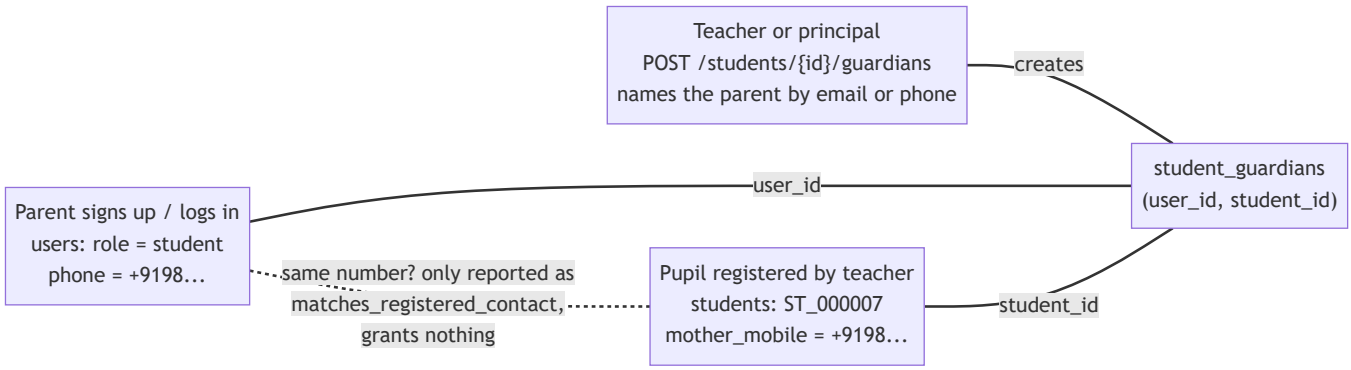
One `users` row per person who signs in, whatever their role. A teacher also has a `teachers` row; a principal and a parent do not. A parent reaches a pupil **only** through `student_guardians`.


users
string user_id PK U_000001
uuid uuid unique
string full_name
string email unique, nullable, lower-cased
string phone unique, nullable, +91... form
string password_hash nullable
string google_id unique, nullable, Google sub claim
string role teacher | student | principal, nullable
bool is_phone_verified
bool is_active
datetime created_at
students
string student_id PK ST_000001
is a
child side
parent side
linked by
teachers
string teacher_id PK TR_000001
uuid uuid unique
string user_id FK unique, one teacher row per user
date date_of_birth nullable
string highest_qualification nullable
string address nullable
string relationship_status nullable
string aadhaar_number unique, nullable, only last 4 ever returned
string emergency_contact_name nullable
string emergency_contact_phone nullable
datetime created_at
datetime updated_at
student_guardians
string user_id PK also FK to users, the parent account
string student_id PK also FK to students, the pupil
string relation Mother, Father, ...
string linked_by_user_id FK staff who made the link
datetime created_at
previous posts
teacher_experience
int id PK
string teacher_id FK
string school_name
string school_address
decimal years Numeric(4,1), so 2.5 survives
datetime created_at

Name and phone are not copied onto `teachers`. They are joined in from `users`, so a profile can never disagree with the account it logs in with.

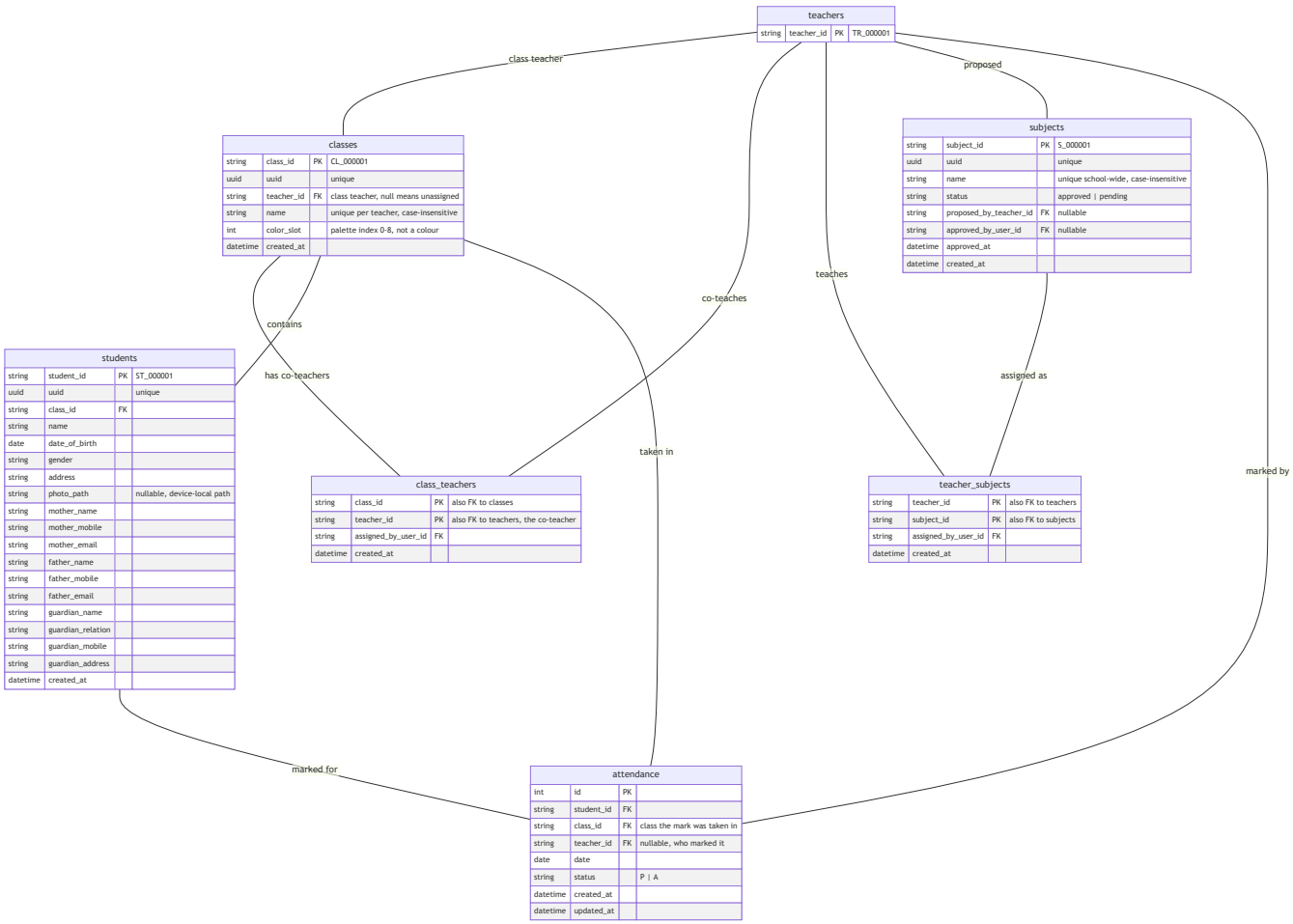
How a parent's login maps to a pupil

The number a parent logs in with (`users.phone`) and the numbers typed on the pupil's record (`students.mother_mobile` etc.) are **never matched automatically**. The only mapping is a `student_guardians` row, and only staff can create one.



---

3. School — classes, pupils, subjects and attendance

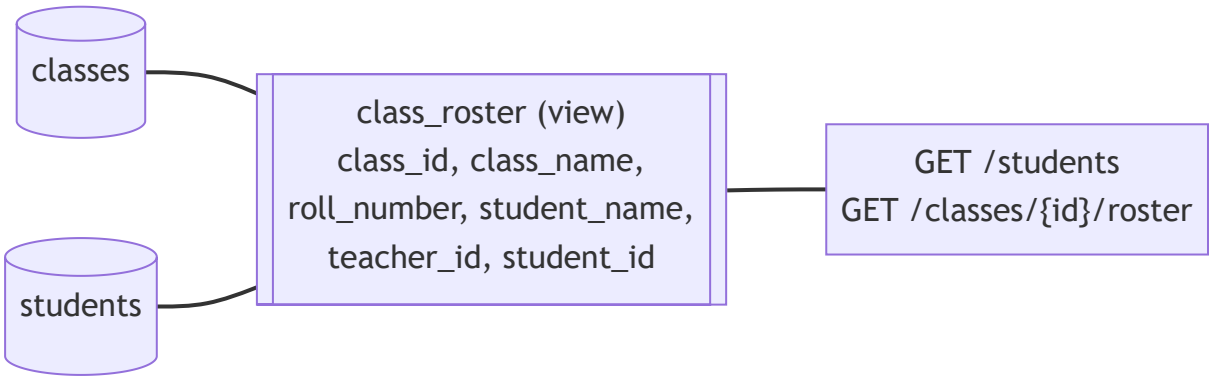


Things the diagram cannot show:

- **A teacher's classes are two halves:** the ones they own (`classes.teacher_id`) plus the ones in `class_teachers` . `app.access.teacher_class_ids` is the one place that unions them.

- **No roll-number column.** Roll numbers are alphabetical within a class and are computed on every read by the `class_roster` view (below).
- **Duplicate pupil rule:** the unique index `uq_students_same_child` covers class + name + date of birth + address + both parents' names, emails and mobiles. The name is in it so that twins are two pupils.
- **One mark per pupil per day:** `uq_attendance_student_date` on `(student_id, date)`. Re-marking a day updates the row.

The `class_roster` view

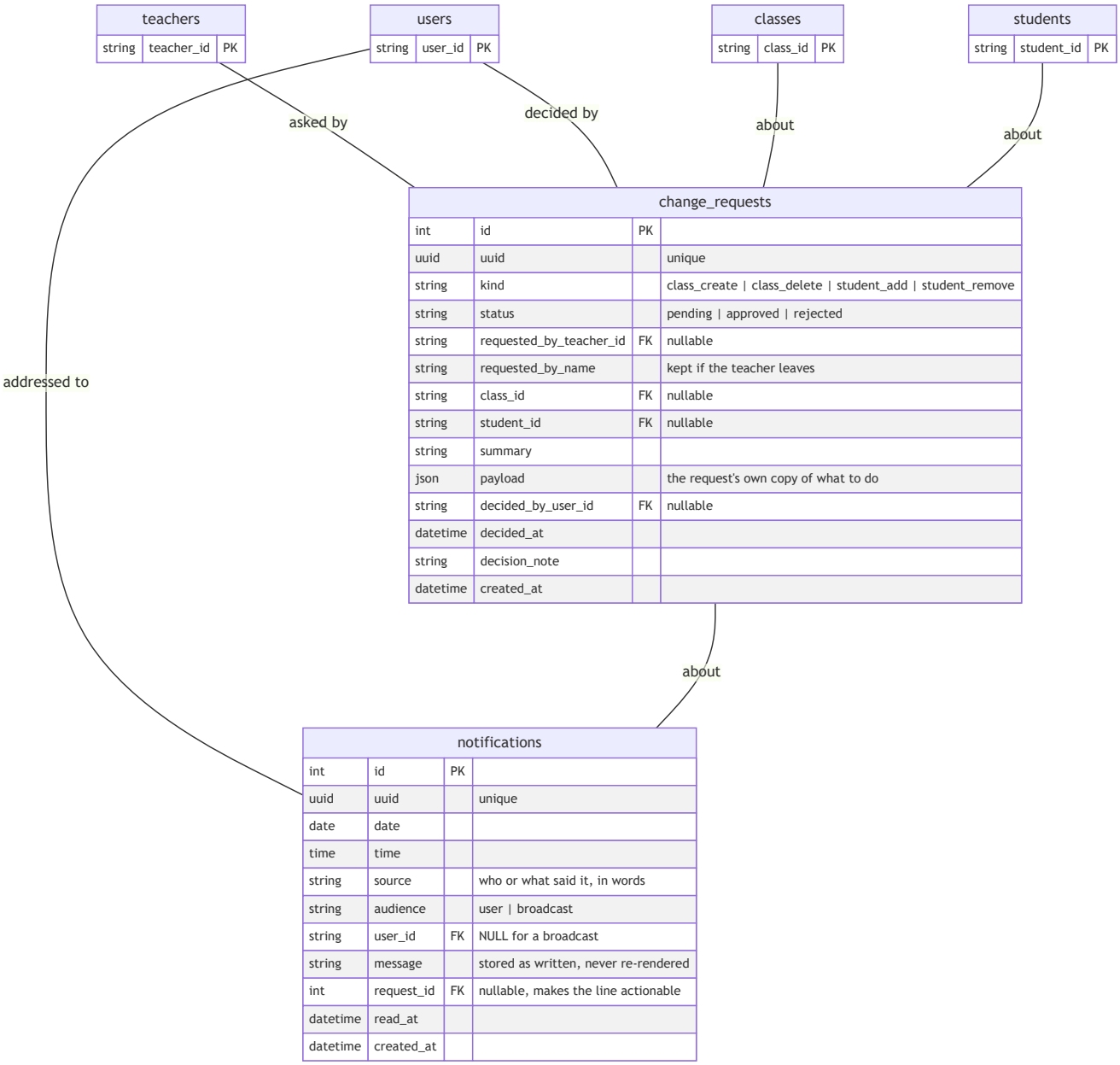


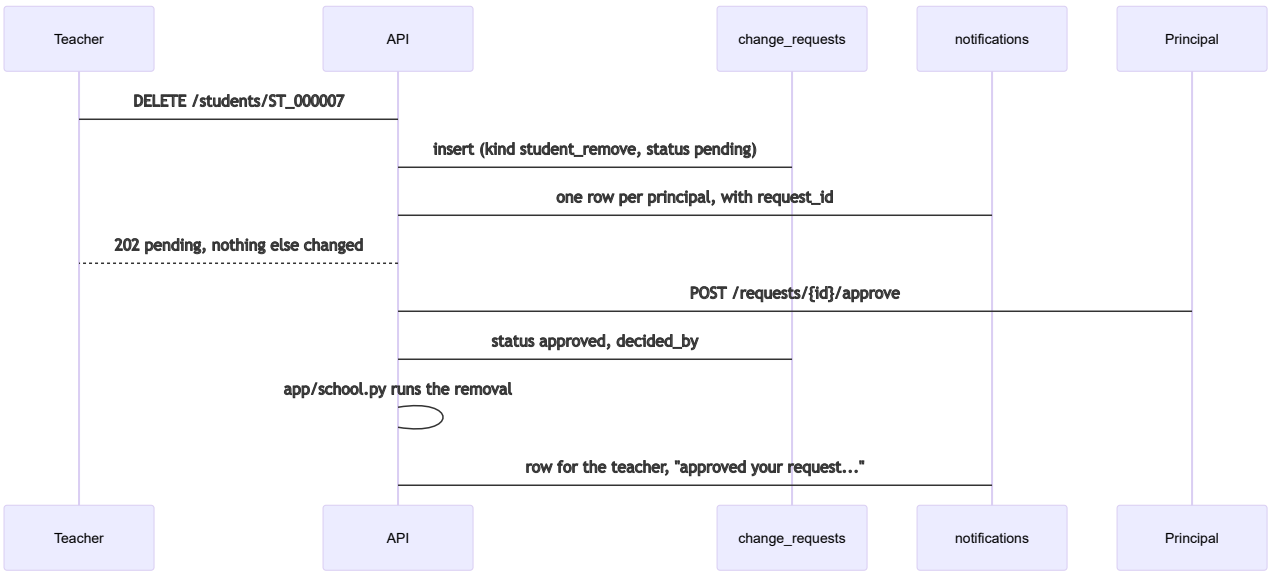
`roll_number` is the pupil's alphabetical position in the class, with `student_id` as the tie-breaker. The view's SQL lives in migration `0001`. `class_roster.teacher_id` is the class **owner** only, so it must not be used to decide what a co-teacher can see.

---

4. Approvals and notifications

A teacher asking to create or delete a class, or to remove a pupil, writes a `change_requests` row and changes nothing else. The principal's answer runs the change. Every line in someone's notification tab is a `notifications` row; a `request_id` on it is what puts Approve and Reject on that line.

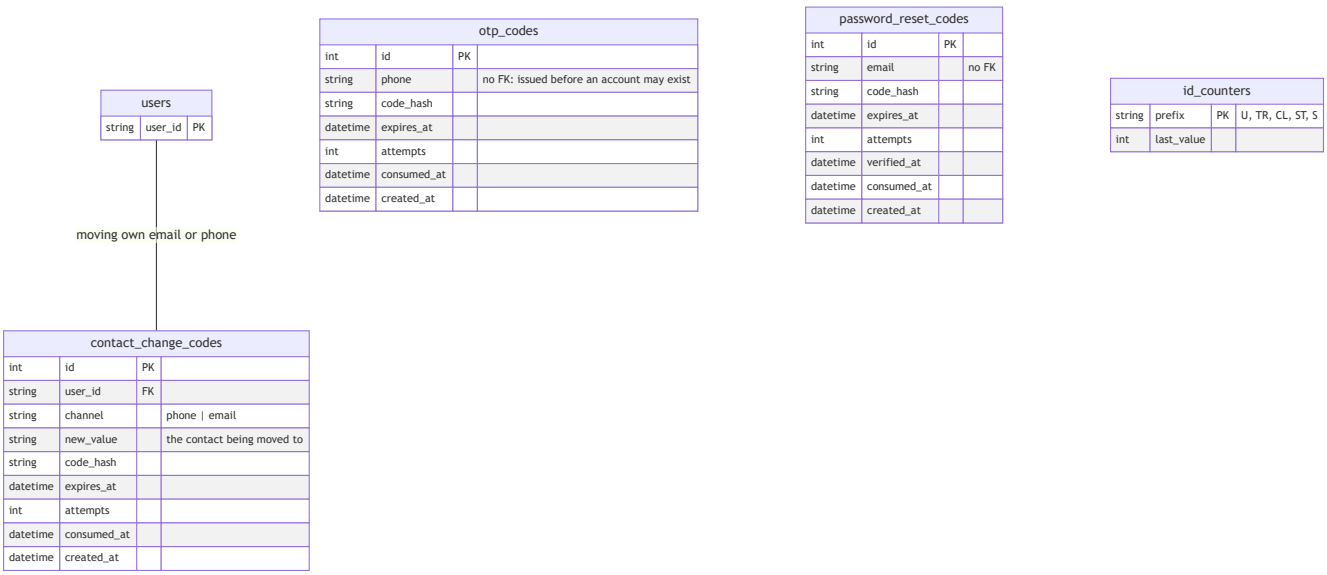

teachers
string teacher_id PK
users
string user_id PK
classes
string class_id PK
students
string student_id PK
asked by
decided by
about
about
addressed to
about
change_requests
int id PK
uuid uuid unique
string kind class_create | class_delete | student_add | student_remove
string status pending | approved | rejected
string requested_by_teacher_id FK nullable
string requested_by_name kept if the teacher leaves
string class_id FK nullable
string student_id FK nullable
string summary
json payload the request's own copy of what to do
string decided_by_user_id FK nullable
datetime decided_at
string decision_note
datetime created_at
notifications
int id PK
uuid uuid unique
date date
time time
string source who or what said it, in words
string audience user | broadcast
string user_id FK NULL for a broadcast
string message stored as written, never re-rendered
int request_id FK nullable, makes the line actionable
datetime read_at
datetime created_at



Which acts need this path (as of 2026-09-27):

Act	Principal	Teacher
Create a class	done outright	request
Delete a class	done outright	request
Register a pupil	done outright	done outright, principals notified
Edit a pupil	done outright	done outright; linked parents notified if a mobile changes
Remove a pupil	done outright	request

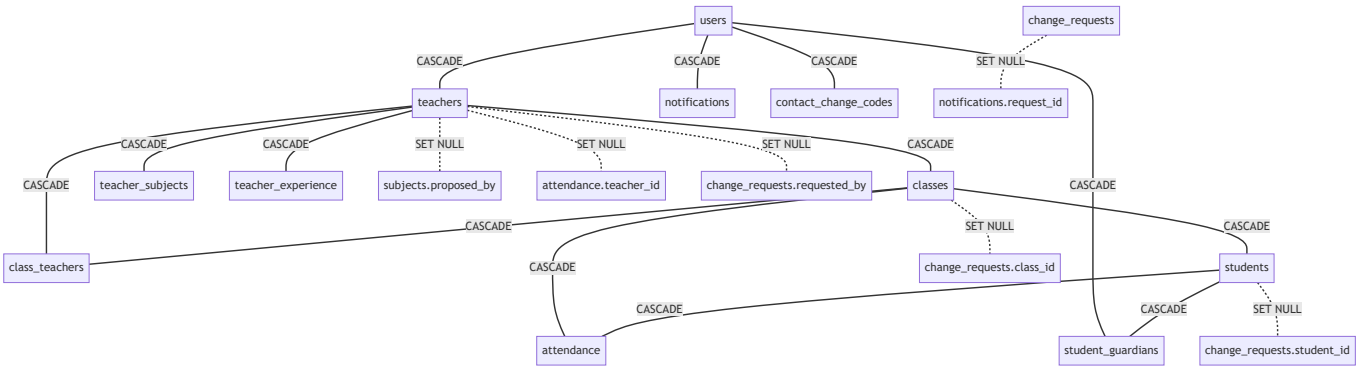
5. Sign-in codes and counters



- **`otp_codes`** is for phone login. A verified code signs in to (or creates) whichever account holds that number.
- **`contact_change_codes`** is kept separate for that reason. It is keyed to one `user_id` and can only move that user's contact; it can never sign anyone in.
- **`password_reset_codes`** exists, but no route uses it yet: password reset is UI only.
- **`id_counters`** holds one row per id prefix. Numbers are never reused, which is what lets a restored class file bring pupils back under their old `ST_` ids.

What deleting a row takes with it

Most foreign keys are `ON DELETE CASCADE`. The `SET NULL` ones keep history alive when a person or thing goes away.



Solid arrows delete the child rows; dotted arrows only blank the column. Every `assigned_by_user_id`, `linked_by_user_id`, `approved_by_user_id` and `decided_by_user_id` is also `SET NULL`.

Deleting a pupil or a class in code also deletes the attendance and guardian rows explicitly (`app/school.py`). PostgreSQL would apply the cascades on its own, but the test suite runs on SQLite, which ignores them unless each connection turns them on.